

1. Python Streaming Generator

A Python-based streaming data generator was developed to simulate real-time telecom transactions. The generator continuously produces CSV files containing telecom call detail records (CDR) and stores them in a shared directory monitored by Apache NiFi. The generated data simulates real-world streaming behavior by producing new files every few seconds.

The generated telecom records included the following fields:

Field	Description
call_id	Unique call identifier
customer_id	Telecom customer identifier
tower_id	Cellular tower identifier
duration	Call duration in seconds
cost	Call cost
call_type	Type of telecom service
timestamp	Transaction timestamp

The generator also intentionally created messy and inconsistent data to simulate real-world data quality issues. The generated errors included:

- Missing values
- Corrupted rows
- Duplicate records
- Negative numeric values
- Different timestamp formats

The Python generator continuously produced CSV files inside the shared folder connected to the NiFi container. This enabled NiFi to automatically detect and ingest files in near real time.

```
PROJECT
├── .venv
├── generator
└── generate_orders.py
    ├── raw_data
    │   ├── telecom_1779026086.csv
    │   ├── telecom_1779026089.csv
    │   ├── telecom_1779026092.csv
    │   ├── telecom_1779026095.csv
    │   ├── telecom_1779026098.csv
    │   ├── telecom_1779026101.csv
    │   ├── telecom_1779026104.csv
    │   ├── telecom_1779026107.csv
    │   ├── telecom_1779026110.csv
    │   ├── telecom_1779027333.csv
    │   ├── telecom_1779027336.csv
    │   ├── telecom_1779027339.csv
    │   ├── telecom_1779027342.csv
    │   ├── telecom_1779027345.csv
    │   ├── telecom_1779027348.csv
    │   ├── telecom_1779027351.csv
    │   ├── telecom_1779028510.csv
    │   ├── telecom_1779028513.csv
    │   ├── telecom_1779028516.csv
    │   ├── telecom_1779028519.csv
    │   ├── telecom_1779028522.csv
    │   ├── telecom_1779028525.csv
    │   ├── telecom_1779028528.csv
    │   ├── telecom_1779028531.csv
    │   ├── telecom_1779028534.csv
    │   └── telecom_1779028537.csv
    ├── OUTLINE
    └── TIMELINE

generator > generate_orders.py > ...
1 import csv
2 import random
3 import time
4 import uuid
5 import os
6 from datetime import datetime
7 from faker import Faker
8
9 fake = Faker()
10
11 OUTPUT_DIR = r"M:\Data Science\nifi-real-time-project\project\raw_data"
12
13 os.makedirs(OUTPUT_DIR, exist_ok=True)
14
15 call_types = ["VOICE", "SMS", "DATA"]
16
17 def random_timestamp():
18     formats = [
19         datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
20         datetime.now().strftime("%d/%m/%Y %H:%M"),
21         datetime.now().strftime("%m-%d-%Y %H:%M:%S"),
22     ]
23     return random.choice(formats)
24
25 PROBLEMS OUTPUT TERMINAL PORTS
26
27 > TERMINAL
28
29 (.venv) PS M:\Data Science\nifi-real-time-project\project\generator> python .\generate_orders.py
Generated: telecom_1779040109.csv
Generated: telecom_1779040112.csv
Generated: telecom_1779040115.csv
Generated: telecom_1779040118.csv
Generated: telecom_1779040121.csv
Generated: telecom_1779040124.csv
Generated: telecom_1779040127.csv
Generated: telecom_1779040130.csv
Generated: telecom_1779040133.csv
Generated: telecom_1779040136.csv
Generated: telecom_1779040139.csv
1 call_id,customer_id,tower_id,duration,cost,call_type,timestamp
60 783936,,TWR30,1406,62.17,VOICE,2026-05-17 16:54:49
61 886496,,TWR15,3281,17.0,SMS,2026-05-17 16:54:49
62 BAD_ROW_@@@@
63 500620,CUST7472,TWR6,404,47.63,SMS,17/05/2026 16:54
64 385547,CUST6085,TWR53,1348,47.68,SMS,17/05/2026 16:54
65 414031,,TWR22,2254,75.87,SMS,2026-05-17 16:54:49
66 366057,CUST9321,TWR38,2662,-50,VOICE,05-17-2026 16:54:49
67 BAD_ROW_@@@@
68 305499,,TWR94,1043,66.01,DATA,17/05/2026 16:54
69 692046,CUST1430,TWR38,1124,41.44,VOICE,2026-05-17 16:54:49
70 701653,,TWR69,2908,75.42,VOICE,05-17-2026 16:54:49
71 260689,,TWR96,3832,26.04,SMS,17/05/2026 16:54
72 BAD_ROW_@@@@
73 104609,CUST6076,TWR75,1852,-50,VOICE,2026-05-17 16:54:49
74 961523,CUST4943,TWR4,2951,-50,VOICE,2026-05-17 16:54:49
75 672622,,TWR96,1268,32.07,SMS,17/05/2026 16:54
76 546532,CUST4431,TWR87,96,51.14,DATA,17/05/2026 16:54
77 490366,CUST1652,TWR90,203,55.62,DATA,05-17-2026 16:54:49
78 672887,CUST2688,TWR78,705,-50,SMS,05-17-2026 16:54:49
79 452025,,TWR75,4581,60.47,SMS,17/05/2026 16:54
80 693289,,TWR80,4118,59.4,VOICE,2026-05-17 16:54:49
81 888551,CUST8165,TWR17,2826,58.05,SMS,05-17-2026 16:54:49
```

2. NiFi Pipeline Design

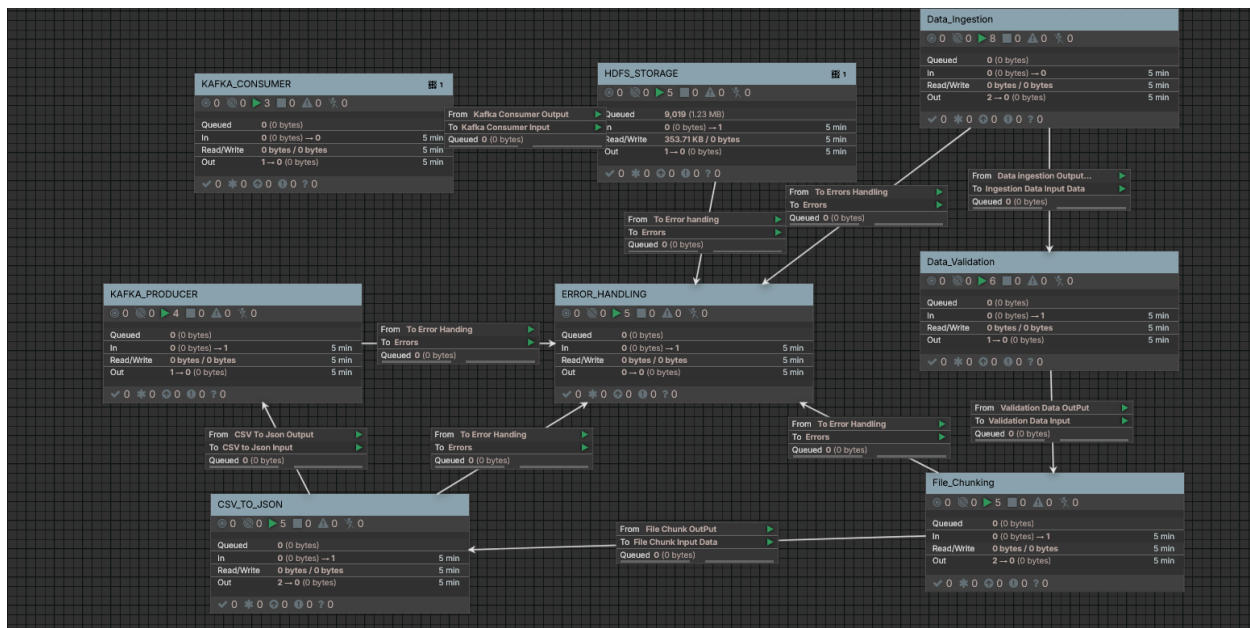
The real-time data pipeline was implemented using [Apache NiFi](#) as the orchestration and ingestion platform. The pipeline was designed using multiple process groups to organize the data flow in a clean and scalable enterprise-style architecture.

The pipeline was divided into the following process groups:

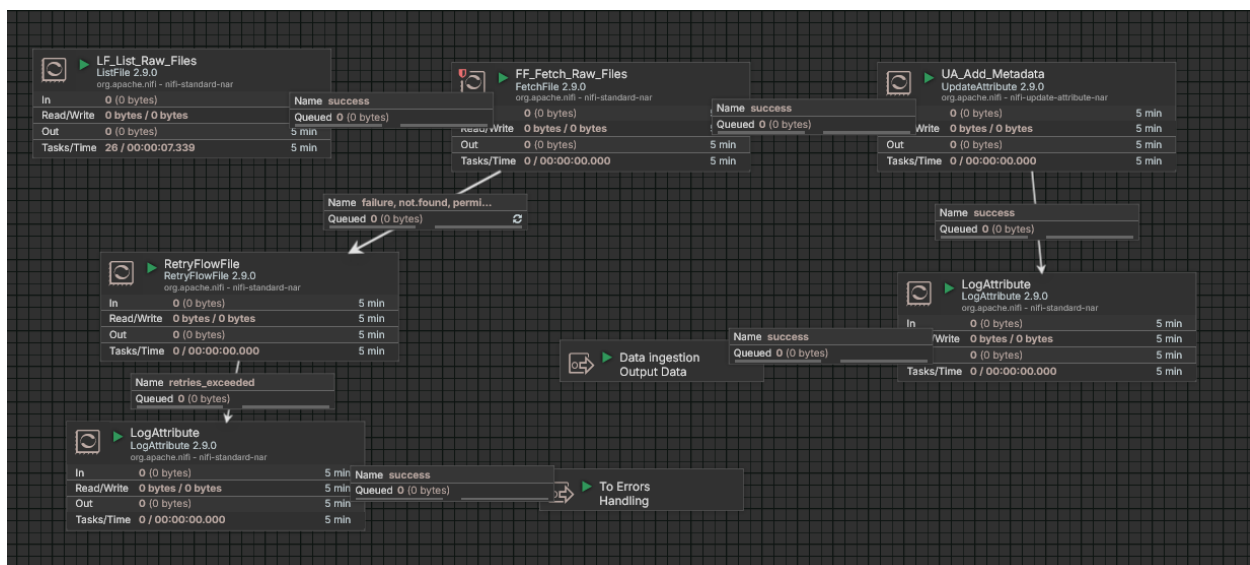
Process Group	Purpose
PG01_File_Ingestion	File detection and ingestion
PG02_Validation	Validation and invalid record filtering
PG03_File_Chunking	Splitting large files into smaller chunks
PG04_CSV_TO_JSON	CSV to JSON transformation
PG05_KAFKA_PRODUCER	Publishing messages to Kafka
PG06_KAFKA_CONSUMER	Consuming Kafka messages
PG07_HDFS_STORAGE	Storing processed data in HDFS
PG08_ERROR_HANDLING	Centralized error management

The pipeline used NiFi FlowFiles to move data between processors. Back pressure, queue management, penalization periods, and provenance tracking were configured to ensure stable and reliable streaming ingestion.

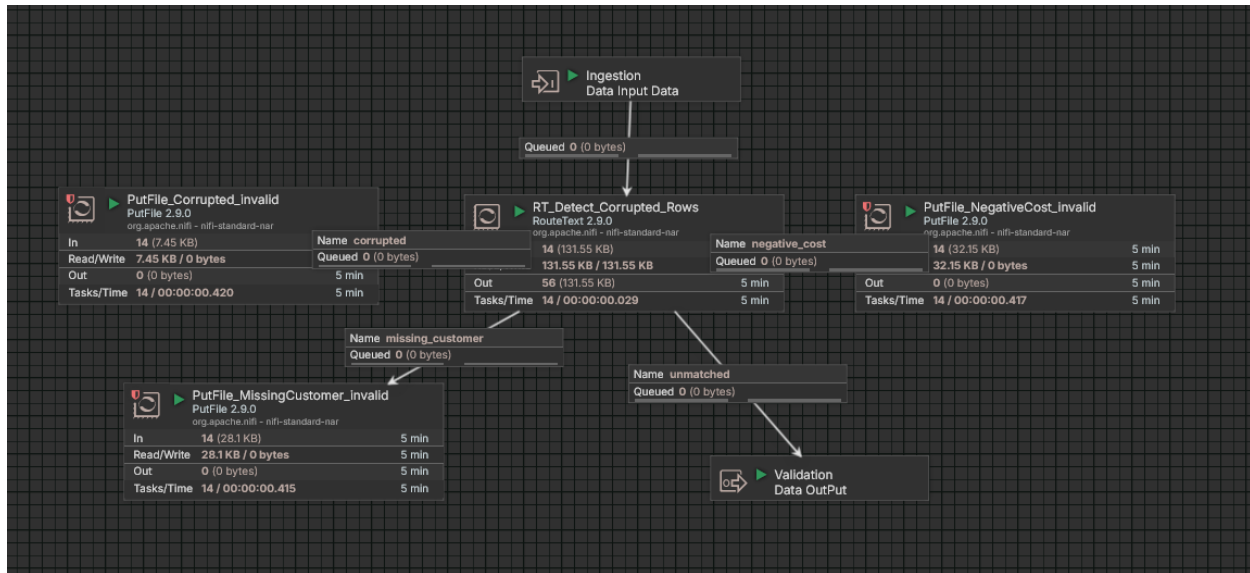
The file ingestion stage used the ListFile and FetchFile processors to continuously monitor the shared directory and ingest newly generated CSV files. Validation processors were then used to filter malformed records before transformation and storage.



File_Ingestion



Validation Group



3. File Chunking

To satisfy the assignment requirement, incoming telecom CSV files were divided into chunks with a maximum size below 64 KB. File chunking was implemented using the SplitText processor inside NiFi.

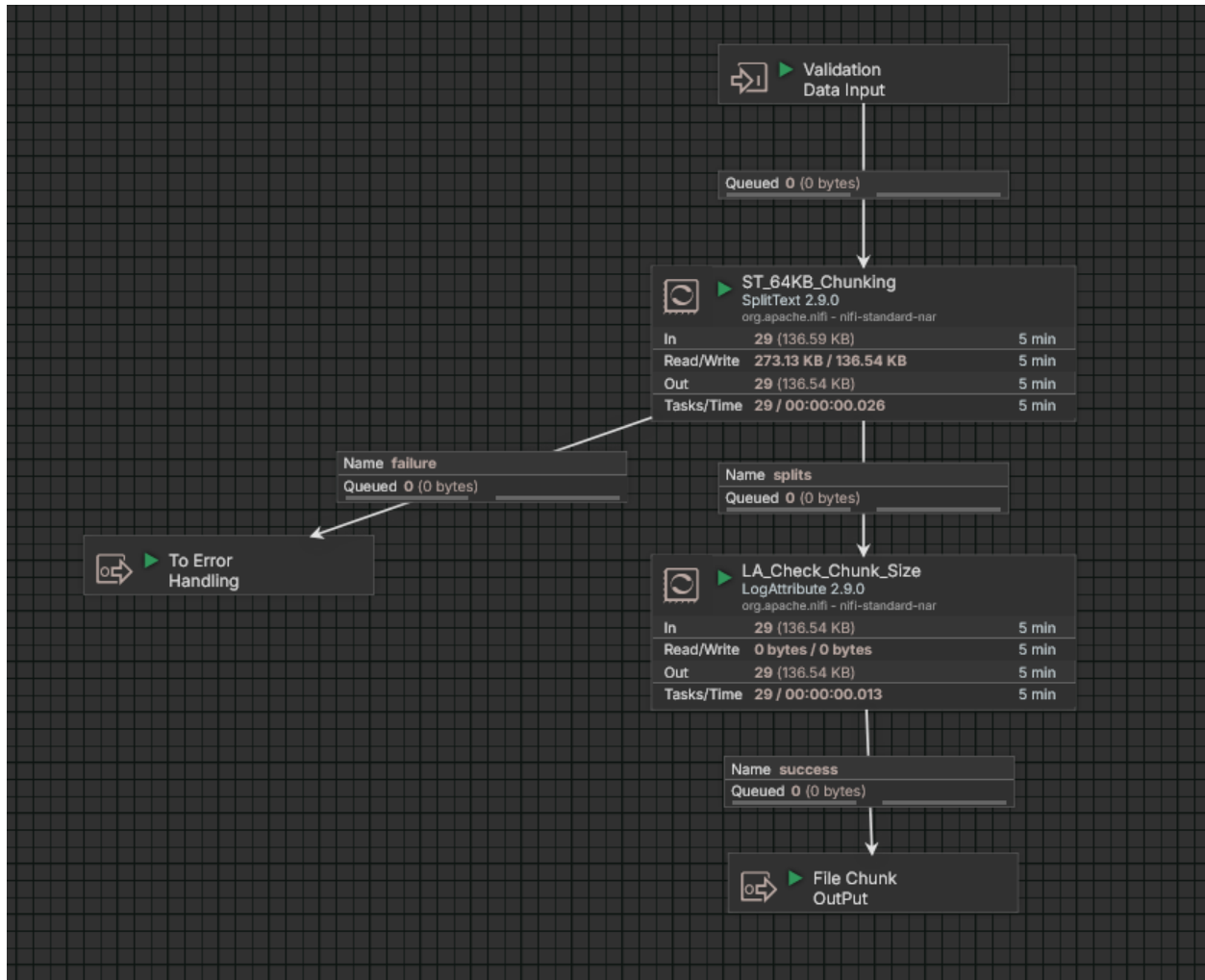
The chunking process divided incoming FlowFiles into smaller fragments while preserving metadata such as:

- fragment.identifier
- fragment.index
- fragment.count
- fragment.size

This ensured that large streaming files could be processed efficiently without overwhelming downstream processors.

The Line Split Count property in the SplitText processor was configured to generate chunk sizes safely below the required threshold.

PG03_File_Chunking



Chunk Attributes

```
file.size
11139

file.source
raw_csv

filename
telecom_1779029213.csv

fragment.count
1

fragment.identifier
b3946e89-550f-4c0b-bde8-d06789c55618

fragment.index
1

fragment.size
5515

ingestion.time
```

4. CSV to JSON Transformation

After validation and chunking, telecom CSV records were transformed into JSON format using NiFi Record processors. The transformation stage used the ConvertRecord processor along with CSVReader and JsonRecordSetWriter controller services.

The CSVReader service automatically inferred the schema from incoming telecom CSV files, while the JsonRecordSetWriter generated structured JSON output.

Example CSV record:

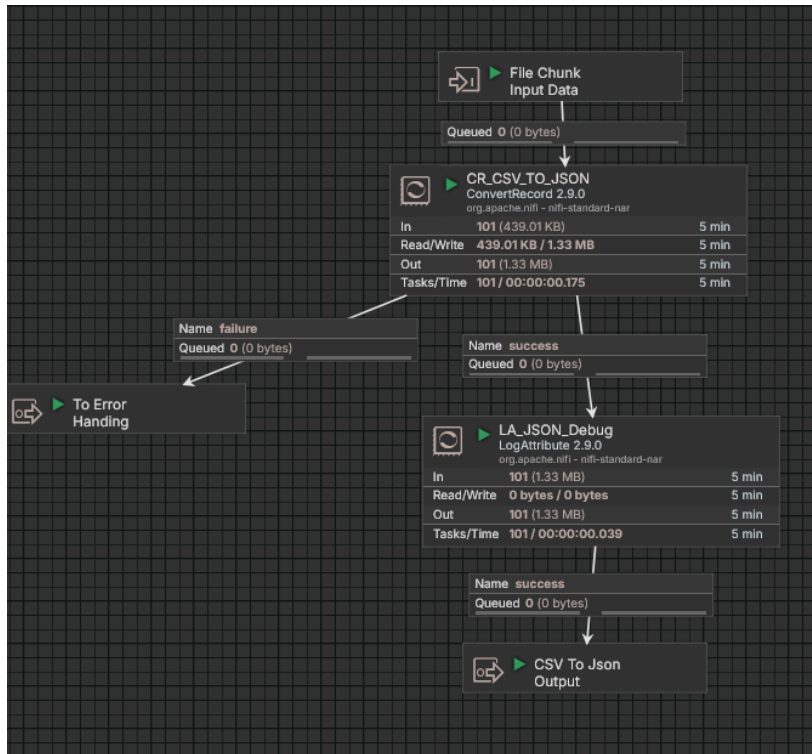
```
1001,CUST01,TWR22,120,4.5,VOICE,2026-05-17 12:00:00
```

Equivalent JSON record:

```
{
  "call_id": 1001,
  "customer_id": "CUST01",
  "tower_id": "TWR22",
  "duration": 120,
  "cost": 4.5,
  "call_type": "VOICE",
  "timestamp": "2026-05-17 12:00:00"
}
```

The transformation stage ensured structured and standardized records before publishing them to Kafka.

CSV_TO_JSON



5. Kafka Integration

[Apache Kafka](#) was used as the streaming messaging platform between the processing and storage layers. Kafka enabled decoupling between data producers and consumers while supporting scalable real-time streaming.

A Kafka topic named telecom-transactions was created with multiple partitions to simulate distributed streaming behavior.

The PublishKafkaRecord_2_6 processor was used to publish transformed JSON records into the Kafka topic. The ConsumeKafkaRecord_2_6 processor was then used to consume streaming messages from Kafka for final storage inside HDFS.

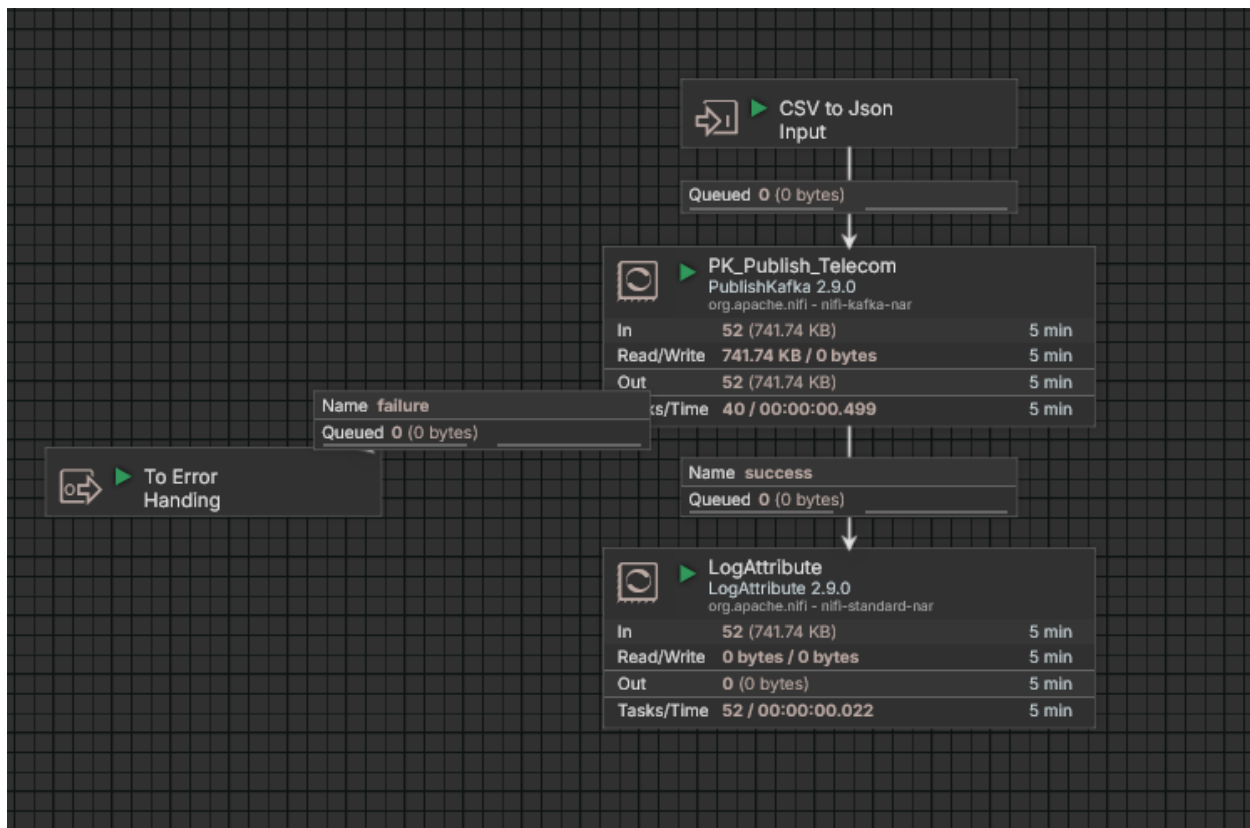
Kafka integration enabled:

- asynchronous streaming
- scalable ingestion
- buffering between systems
- reliable message delivery

Kafka Topic Creation

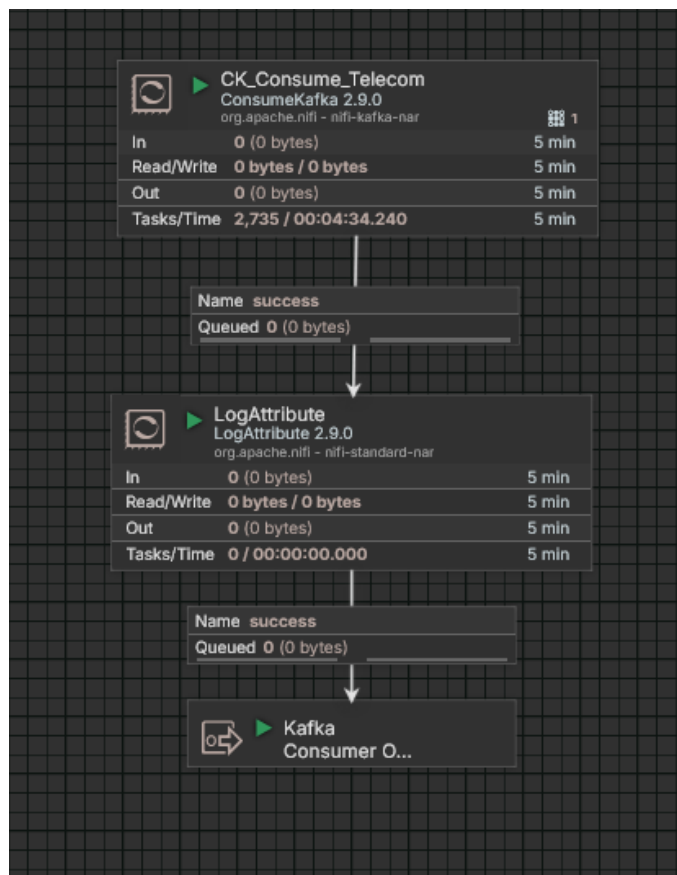
```
[appuser@kafka1 ~]$ kafka-topics --bootstrap-server kafka1:19092 \  
> --create \  
> --topic telecom-transactions \  
> --partitions 6 \  
> --replication-factor 2 \  
> --config retention.ms=604800000 \  
> --config compression.type=snappy \  
Created topic telecom-transactions.  
  
... 0 more  
[appuser@kafka1 ~]$ kafka-topics --list --bootstrap-server kafka1:19092  
banking-transactions  
banking-transactions-dlq  
telecom-transactions  
[appuser@kafka1 ~]$
```

Kafka Producer Group



```
[appuser@kafka1 ~]$ kafka-console-consumer \
> --topic telecom-transactions \
> --bootstrap-server kafka1:19092 \
> --from-beginning
{"call_id":372590,"customer_id":"CUST3560","tower_id":"TWR51","duration":2480,"cost":50.2,"call_type":"DATA","timestamp":"2026-05-17 18:06:42"}
{"call_id":490562,"customer_id":"CUST5236","tower_id":"TWR97","duration":1407,"cost":65.47,"call_type":"VOICE","timestamp":"17/05/2026 18:06"}
{"call_id":752998,"customer_id":"CUST8215","tower_id":"TWR21","duration":4303,"cost":54.37,"call_type":"VOICE","timestamp":"17/05/2026 18:06"}
{"call_id":704751,"customer_id":"CUST9852","tower_id":"TWR33","duration":2069,"cost":4.74,"call_type":"SMS","timestamp":"05-17-2026 18:06:42"}
{"call_id":722592,"customer_id":"CUST8561","tower_id":"TWR93","duration":3839,"cost":20.16,"call_type":"DATA","timestamp":"2026-05-17 18:06:42"}
{"call_id":937419,"customer_id":"CUST5396","tower_id":"TWR4","duration":4288,"cost":98.02,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":866627,"customer_id":"CUST9310","tower_id":"TWR21","duration":4751,"cost":8.83,"call_type":"SMS","timestamp":"2026-05-17 18:06:42"}
{"call_id":401269,"customer_id":"CUST8817","tower_id":"TWR85","duration":2929,"cost":99.2,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":316003,"customer_id":"CUST8591","tower_id":"TWR71","duration":2758,"cost":55.46,"call_type":"SMS","timestamp":"05-17-2026 18:06:42"}
{"call_id":247075,"customer_id":"CUST8999","tower_id":"TWR42","duration":4192,"cost":13.61,"call_type":"SMS","timestamp":"05-17-2026 18:06:42"}
{"call_id":271490,"customer_id":"CUST8356","tower_id":"TWR8","duration":1279,"cost":15.74,"call_type":"DATA","timestamp":"05-17-2026 18:06:42"}
{"call_id":624328,"customer_id":"CUST3771","tower_id":"TWR87","duration":3779,"cost":90.44,"call_type":"VOICE","timestamp":"17/05/2026 18:06"}
{"call_id":240562,"customer_id":"CUST7579","tower_id":"TWR43","duration":3449,"cost":63.91,"call_type":"VOICE","timestamp":"17/05/2026 18:06"}
{"call_id":664746,"customer_id":"CUST7017","tower_id":"TWR47","duration":237,"cost":33.35,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":469196,"customer_id":"CUST1834","tower_id":"TWR37","duration":4125,"cost":63.33,"call_type":"DATA","timestamp":"2026-05-17 18:06:42"}
{"call_id":324036,"customer_id":"CUST6378","tower_id":"TWR79","duration":4430,"cost":10.93,"call_type":"SMS","timestamp":"2026-05-17 18:06:42"}
{"call_id":727283,"customer_id":"CUST4726","tower_id":"TWR68","duration":2156,"cost":53.68,"call_type":"VOICE","timestamp":"05-17-2026 18:06:42"}
{"call_id":930151,"customer_id":"CUST5659","tower_id":"TWR9","duration":790,"cost":96.27,"call_type":"DATA","timestamp":"05-17-2026 18:06:42"}
{"call_id":545293,"customer_id":"CUST3186","tower_id":"TWR80","duration":797,"cost":27.07,"call_type":"DATA","timestamp":"05-17-2026 18:06:42"}
{"call_id":185247,"customer_id":"CUST7329","tower_id":"TWR13","duration":4488,"cost":27.71,"call_type":"VOICE","timestamp":"2026-05-17 18:06:42"}
{"call_id":245668,"customer_id":"CUST9527","tower_id":"TWR84","duration":1573,"cost":12.53,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":371966,"customer_id":"CUST3403","tower_id":"TWR5","duration":2629,"cost":54.0,"call_type":"SMS","timestamp":"2026-05-17 18:06:42"}
{"call_id":818536,"customer_id":"CUST5197","tower_id":"TWR17","duration":1338,"cost":99.78,"call_type":"DATA","timestamp":"05-17-2026 18:06:42"}
{"call_id":890140,"customer_id":"CUST4560","tower_id":"TWR78","duration":1072,"cost":65.14,"call_type":"DATA","timestamp":"05-17-2026 18:06:42"}
{"call_id":583363,"customer_id":"CUST3582","tower_id":"TWR47","duration":4075,"cost":91.91,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":813126,"customer_id":"CUST4088","tower_id":"TWR99","duration":1965,"cost":43.92,"call_type":"SMS","timestamp":"17/05/2026 18:06"}
{"call_id":940325,"customer_id":"CUST7629","tower_id":"TWR71","duration":2724,"cost":95.36,"call_type":"VOICE","timestamp":"2026-05-17 18:06:42"}
{"call_id":784656,"customer_id":"CUST3751","tower_id":"TWR31","duration":1403,"cost":82.08,"call_type":"SMS","timestamp":"2026-05-17 18:06:42"}
{"call_id":974149,"customer_id":"CUST2605","tower_id":"TWR81","duration":2005,"cost":12.22,"call_type":"DATA","timestamp":"2026-05-17 18:06:42"}
{"call_id":450379,"customer_id":"CUST2913","tower_id":"TWR43","duration":4284,"cost":77.5,"call_type":"SMS","timestamp":"17/05/2026 18:06"}
{"call_id":226737,"customer_id":"CUST9728","tower_id":"TWR77","duration":2784,"cost":43.29,"call_type":"DATA","timestamp":"2026-05-17 18:06:42"}
{"call_id":121389,"customer_id":"CUST2756","tower_id":"TWR2","duration":3874,"cost":10.08,"call_type":"VOICE","timestamp":"17/05/2026 18:06"}
{"call_id":916555,"customer_id":"CUST4033","tower_id":"TWR17","duration":431,"cost":27.45,"call_type":"VOICE","timestamp":"05-17-2026 18:06:42"}
{"call_id":934061,"customer_id":"CUST8120","tower_id":"TWR92","duration":111,"cost":98.28,"call_type":"DATA","timestamp":"17/05/2026 18:06"}
{"call_id":101358,"customer_id":"CUST5673","tower_id":"TWR63","duration":2714,"cost":97.58,"call_type":"VOICE","timestamp":"05-17-2026 18:06:42"}
```

Kafka consumer:



6. HDFS Storage

The final processed telecom JSON records were stored inside [Apache Hadoop](#) Distributed File System (HDFS). Storage was implemented using the PutHDFS processor in NiFi.

To improve scalability and organization, dynamic HDFS partitioning was implemented using year, month, and day partitions. The partitioning strategy generated the following structure:

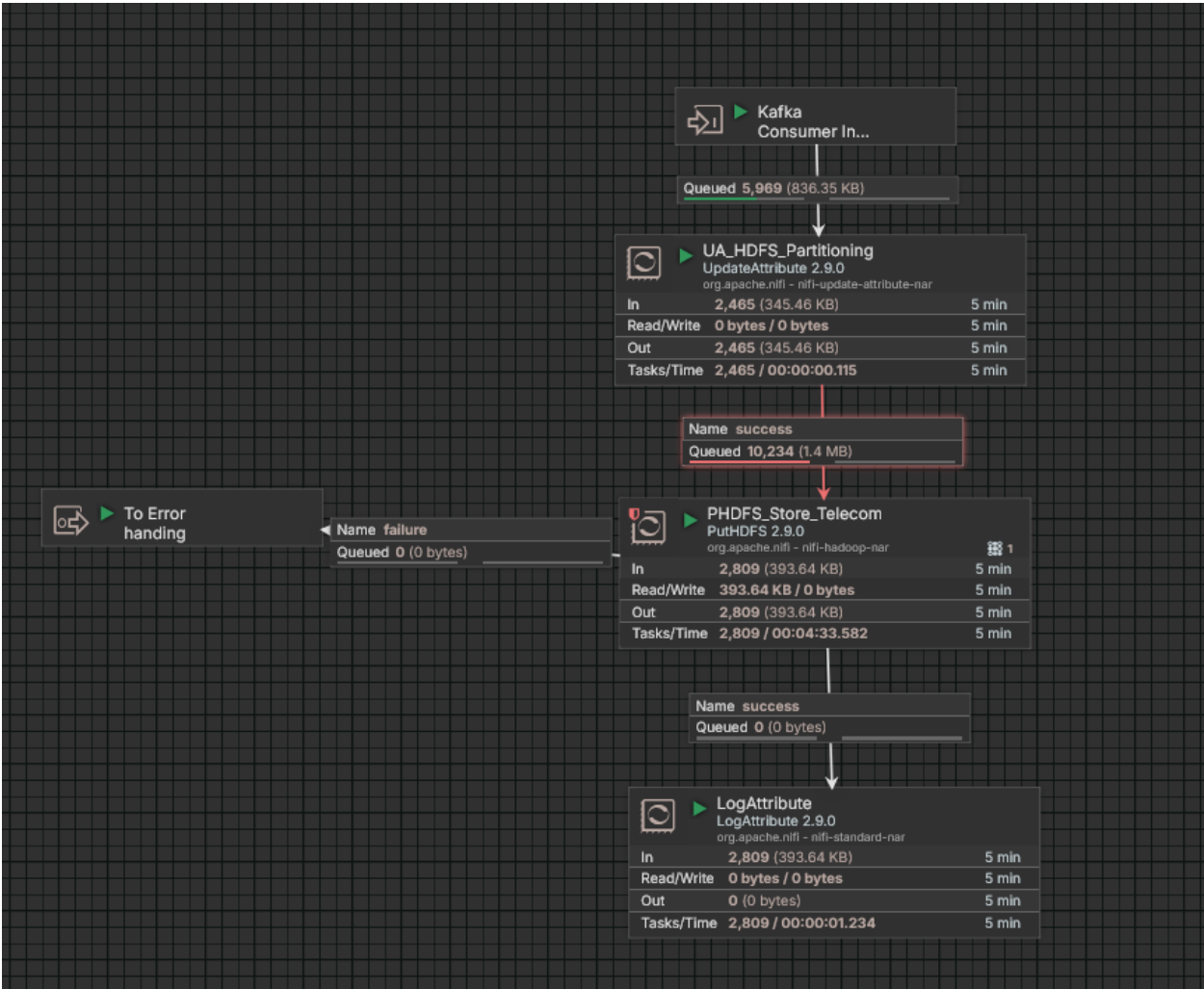
```
/data/year=2026/month=05/day=17
```

The UpdateAttribute processor dynamically generated partition attributes using NiFi Expression Language. These attributes were then used by PutHDFS to write records into the appropriate HDFS directory.

This partitioning strategy improves:

- scalability
- organization
- future querying performance

HDFS_STORAGE



HDFS Directory Structure

```
itversity@itvdelab:~$ hdfs dfs -ls /user/nifi/data
Found 937 items
-rw-r--r-- 1 nifi nifi      145 2026-05-17 18:11 /user/nifi/data/.telecom_1779040349359.json
-rw-r--r-- 1 nifi nifi      139 2026-05-17 17:18 /user/nifi/data/telecom_1779036620992.json
-rw-r--r-- 1 nifi nifi      144 2026-05-17 17:18 /user/nifi/data/telecom_1779036620993.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:18 /user/nifi/data/telecom_1779036620994.json
-rw-r--r-- 1 nifi nifi      146 2026-05-17 17:18 /user/nifi/data/telecom_1779036620995.json
-rw-r--r-- 1 nifi nifi      143 2026-05-17 17:18 /user/nifi/data/telecom_1779036620996.json
-rw-r--r-- 1 nifi nifi      142 2026-05-17 17:18 /user/nifi/data/telecom_1779036620997.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:18 /user/nifi/data/telecom_1779036620998.json
-rw-r--r-- 1 nifi nifi      144 2026-05-17 17:18 /user/nifi/data/telecom_1779036621015.json
-rw-r--r-- 1 nifi nifi      141 2026-05-17 17:19 /user/nifi/data/telecom_1779036621016.json
-rw-r--r-- 1 nifi nifi      139 2026-05-17 17:19 /user/nifi/data/telecom_1779036621017.json
-rw-r--r-- 1 nifi nifi      143 2026-05-17 17:19 /user/nifi/data/telecom_1779036621018.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:19 /user/nifi/data/telecom_1779036621019.json
-rw-r--r-- 1 nifi nifi      143 2026-05-17 17:19 /user/nifi/data/telecom_1779036621020.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:19 /user/nifi/data/telecom_1779036621047.json
-rw-r--r-- 1 nifi nifi      141 2026-05-17 17:19 /user/nifi/data/telecom_1779036621048.json
-rw-r--r-- 1 nifi nifi      142 2026-05-17 17:19 /user/nifi/data/telecom_1779036621049.json
-rw-r--r-- 1 nifi nifi      141 2026-05-17 17:20 /user/nifi/data/telecom_1779036621050.json
-rw-r--r-- 1 nifi nifi      146 2026-05-17 17:20 /user/nifi/data/telecom_1779036621051.json
-rw-r--r-- 1 nifi nifi      146 2026-05-17 17:20 /user/nifi/data/telecom_1779036621066.json
-rw-r--r-- 1 nifi nifi      143 2026-05-17 17:20 /user/nifi/data/telecom_1779036621067.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:20 /user/nifi/data/telecom_1779036621068.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:20 /user/nifi/data/telecom_1779036621074.json
-rw-r--r-- 1 nifi nifi      139 2026-05-17 17:20 /user/nifi/data/telecom_1779036621075.json
-rw-r--r-- 1 nifi nifi      141 2026-05-17 17:20 /user/nifi/data/telecom_1779036621076.json
-rw-r--r-- 1 nifi nifi      141 2026-05-17 17:20 /user/nifi/data/telecom_1779036621077.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:20 /user/nifi/data/telecom_1779036621094.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:20 /user/nifi/data/telecom_1779036621095.json
-rw-r--r-- 1 nifi nifi      145 2026-05-17 17:20 /user/nifi/data/telecom_1779036621096.json
-rw-r--r-- 1 nifi nifi      144 2026-05-17 17:20 /user/nifi/data/telecom_1779036621097.json
-rw-r--r-- 1 nifi nifi      142 2026-05-17 17:20 /user/nifi/data/telecom_1779036621103.json
-rw-r--r-- 1 nifi nifi      140 2026-05-17 17:20 /user/nifi/data/telecom_1779036621104.json
-rw-r--r-- 1 nifi nifi      142 2026-05-17 17:20 /user/nifi/data/telecom_1779036621105.json
-rw-r--r-- 1 nifi nifi      146 2026-05-17 17:20 /user/nifi/data/telecom_1779036621106.json
```

7. Monitoring and Reliability

To ensure stable real-time streaming behavior, several reliability and monitoring mechanisms were implemented inside NiFi.

Back pressure thresholds were configured on all queues to prevent memory exhaustion and uncontrolled FlowFile accumulation during heavy ingestion. Yield duration and penalization periods were also configured to prevent aggressive retries during temporary failures.

The pipeline also implemented centralized error handling using a dedicated error handling process group. Failed FlowFiles from Kafka, HDFS, and transformation processors were routed into a dead-letter queue for troubleshooting and recovery.

NiFi provenance tracking and queue monitoring were used to observe FlowFile movement throughout the pipeline. Provenance events such as RECEIVE, FETCH, SPLIT, SEND, and PUT were used to trace the lifecycle of streaming data.

These monitoring and reliability features improved:

- pipeline stability

- fault tolerance
- operational visibility
- failure recovery

Error Handling Group

